

14 Cache and Persist

Cache and Persist in Spark

Why do we need caching?

Spark is **lazy**.

If the same DataFrame is used multiple times, Spark will:

- Recompute it again and again
- Read data again from source
- Repeat transformations

Caching avoids this by storing intermediate results in memory or disk.

cache()

What is cache?

`cache()` is a **shortcut** for `persist()` with default storage level.

```
df.cache()
```

Equivalent to:

```
df.persist(StorageLevel.MEMORY_AND_DISK)
```

Important points about cache()

- Lazy operation (nothing happens immediately)
 - Data is cached **only after an action**
 - Caches **only the partitions that are materialized**
 - Default storage level: **MEMORY_AND_DISK**
-

Example

```
df.cache()
df.count() # triggers caching of all partitions
```

persist()

What is persist?

`persist()` gives **full control** over how data is stored.

```
from pyspark import StorageLevel

df.persist(StorageLevel.MEMORY_ONLY)
```

Common Storage Levels

Storage Level	Meaning
MEMORY_ONLY	Store only in memory
MEMORY_AND_DISK	Spill to disk if memory is insufficient
DISK_ONLY	Store only on disk
MEMORY_ONLY_SER (Only in scala)	Serialized in memory (less memory, more CPU)
MEMORY_AND_DISK_SER(only in scala)	Serialized + disk spill
MEMORY_ONLY_2	creates 2 replicas only in memory

When to use persist instead of cache?

Use `persist()` when:

- Data is very large
- Memory is limited
- You want predictable behavior
- You want to avoid recomputation cost vs memory tradeoffs

What happens with **MEMORY_ONLY** when data does NOT fit in memory?

```
df.persist(StorageLevel.MEMORY_ONLY)
```

With **MEMORY_ONLY** :

- Spark tries to cache each partition in memory
- If a partition **fits**, it is cached
- If a partition **does not fit**, it is:
 - **Not cached**
 - **Dropped immediately**

Spark does **not**:

- Spill to disk
- Throw an error
- Block execution

What happens next time that partition is needed?

Spark will:

- Recompute the partition from lineage/DAG
- Read data again from source
- Re-run transformations

This can cause:

- Slower performance
- Unexpected recomputation
- High CPU usage

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
```

```

.master("spark://spark-master:7077")
.appName("sparksql")
.config("spark.sql.warehouse.dir", "/data/spark-warehouse")
.config("spark.executor.memory", "512m")
.getOrCreate()
)

from pyspark import StorageLevel

orders_schema =

orders_df1 = (
    spark.read
        .format("csv")
        .option("header", True)
        .schema(orders_schema)
        .load("/data/orders_1gb.csv")
)
orders_df1.persist(StorageLevel.MEMORY_ONLY) → memory only
#orders_df1.cache() → cached in memory and disk
#orders_df1.persist(StorageLevel.DISK_ONLY) → only in disk
orders_df1.count()

# instant result # see UI DAG
orders_df1.filter("order_id=1").show()

df_cached = orders_df1.filter("order_id =1")
df_cached.count()

```

unpersist()

Why unpersist?

Cached data:

- Occupies executor memory
- Can cause memory pressure

Free it when no longer needed.

```
df.unpersist()
```

Cache a table :

Step 1: Create database and table

```
spark.sql("CREATE DATABASE IF NOT EXISTS mydb")
spark.sql("USE mydb")
orders_df.write \
    .mode("overwrite") \
    .format("parquet") \
    .saveAsTable("orders")
```

Step 2: Cache the Managed Table using `spark.sql()`

```
spark.sql("CACHE TABLE orders")

#lazy
spark.sql("CACHE LAZY TABLE orders")

spark.sql("select * from orders").write
```

Step 3: Trigger caching

```
spark.sql("SELECT COUNT(*) FROM orders").show()
```

Now:

- Full table is read
- All partitions are cached

Step 4: Verify cache

```
spark.catalog.isCached("orders")
```

Step 5: Query cached managed table

```
spark.sql("""  
SELECT category, SUM(sales) AS total_sales  
FROM orders  
GROUP BY category  
""").show()
```

This query:

- Reads from cache
- Avoids disk IO
- Much faster on repeated runs

Step 6: Uncache the table

```
spark.sql("UNCACHE TABLE orders")
```

Best practices

- Cache only if reused multiple times
 - Cache after expensive transformations (joins, aggregations)
 - Trigger full action like `count()` after cache
 - Monitor Storage tab in Spark UI
 - Always `unpersist()` when done
-